Motilal Nehru National Institute of Technology Allahabad

Prayagraj, Uttar Pradesh: 211004

Development of a Computer Vision Framework for Motion Feature Extraction of a Projectile using Physics-Informed Neural Networks

Group Project Technical Report

Submitted by

Abhijit Saha	**Shivam Raj Srivastav**	**Vidhi Singh**
20236004	20230051	20238027

Group 7

Under the Guidance of

Dr. Abhishek Kumar Tiwari

Contents

1 Introduction

Reconstructing the flight path of a projectile from a standard video recording is a deceptively challenging problem. The camera provides only 2D pixel coordinates, the image is contaminated by background clutter and player motion, and the underlying physics involves non-linear aerodynamic forces that have no convenient closed-form solution.

The **CARS** (Computer-Assisted Reconstitution of Shots) pipeline addresses this challenge through a tightly integrated four-stage approach:

1. **Computer Vision**: Automatic per-frame detection of the ball using background subtraction and shape-based filtering, producing a raw time-stamped pixel trajectory.
2. **Data Cleaning**: Removal of false detections (player limbs, shadows) via an interactive lasso tool or automated RANSAC-based filtering.
3. **Physics-Informed Neural Network (PINN)**: A neural network that learns the trajectory while simultaneously satisfying the aerodynamic differential equations governing drag and Magnus spin lift, enabling identification of physical quantities like drag coefficient C_D and spin rate ω.
4. **Comparative Evaluation**: The PINN result is benchmarked against two classical methods (Kalman Filter and Parabolic Regression) using standard trajectory error metrics.

The pipeline was validated across **12 independent experiments** covering multiple rally shots and serve conditions, all recorded at 1920×1080 @ 30 fps. Object types supported include: Tennis Ball, Table Tennis, Football, Basketball, Cricket Ball, Baseball, Volleyball, Mortar Shell, and a Generic object.

1.1 Motivation

While the system is demonstrated on sports footage, the underlying problem of reconstructing a ballistic trajectory from monocular video has direct relevance to **defence applications**. A few concrete use cases drove the design choices:

- **Threat assessment**: Knowing the speed and drag-corrected trajectory of an incoming projectile in real time allows an immediate estimate of its danger level and likely impact zone, without relying on radar.
- **Launch origin reconstruction**: Post-event, the fitted trajectory can be extrapolated backward in time to narrow down the likely launch point, which is useful for counter-battery analysis.
- **Uncertainty-aware impact prediction**: Because the PINN is a continuous function, confidence intervals on its output can be propagated to produce a probabilistic impact zone rather than a single predicted point.
- **Command support**: Extracted kinematic parameters (speed, spin, drag coefficient) give analysts a compact, physics-grounded summary of observed projectile behaviour that feeds into broader threat models.

The sports setting provided a controlled, repeatable environment for developing and validating the pipeline before deployment in more operationally sensitive contexts.

2 Background: Physics of Projectile Flight

2.1 Classical (Vacuum) Motion

In an idealised vacuum, a projectile launched with speed V_0 at angle θ follows a perfect parabolic arc governed by:

$$x(t) = V_0 \cos(\theta) \cdot t, \quad y(t) = V_0 \sin(\theta) \cdot t - \frac{1}{2} g t^2 \tag{1}$$

where $g = 9.81\text{m/s}^2$. This model is analytically simple but insufficient for real sports balls, where aerodynamic effects are substantial.

2.2 Aerodynamic Drag

Moving through air of density $\rho = 1.2\text{kg/m}^3$, a sphere of mass m, radius r, and drag coefficient C_D experiences a retarding force:

$$\boldsymbol{F}_{\text{drag}} = -\frac{1}{2} \rho C_D \pi r^2 \, |\boldsymbol{v}| \, \boldsymbol{v} \tag{2}$$

Defining the aerodynamic scale factor $K = \rho \pi r^2 / (2m)$, the equations of motion become:

$$\ddot{x} = -K C_D \, |\boldsymbol{v}| \, \dot{x}, \quad \ddot{y} = -g - K C_D \, |\boldsymbol{v}| \, \dot{y} \tag{3}$$

where $|\boldsymbol{v}| = \sqrt{\dot{x}^2 + \dot{y}^2}$ is the instantaneous speed.

2.3 Magnus Spin Lift

A spinning ball generates a lateral Magnus force perpendicular to its velocity. The spin lift coefficient is:

$$C_L = \frac{1}{2 + (r\omega / \, |\boldsymbol{v}|)^{-1}} \tag{4}$$

Adding the Magnus terms, the full coupled ODE system is:

$$\ddot{x} = -K C_D \, |\boldsymbol{v}| \, \dot{x} + K \, |\boldsymbol{v}| \, \frac{C_L}{\omega} (\omega \dot{y}) \tag{5}$$

$$\ddot{y} = -g - K C_D \, |\boldsymbol{v}| \, \dot{y} - K \, |\boldsymbol{v}| \, \frac{C_L}{\omega} (\omega \dot{x}) \tag{6}$$

This system has no closed-form solution for arbitrary C_D and ω, which is the core motivation for the PINN approach described in Section 4.

3 Computer Vision Detection Pipeline

The detection pipeline converts a raw video into a sequence of time-stamped metric positions (t_i, x_i, y_i).

3.1 Step 1: MOG2 Background Subtraction

The first step separates the moving ball from the static background. The Gaussian Mixture of Gaussians (MOG2) algorithm builds an adaptive statistical model of the background by observing each pixel over a history of 500 frames. Pixels that deviate significantly from their learned background distribution are flagged as **foreground**. This produces a binary foreground mask at each frame.

The key advantage of MOG2 is its ability to adapt to gradual scene changes (e.g., lighting shifts, crowd movement) while still reacting quickly to fast-moving objects like a tennis ball. The foreground mask is used to gate subsequent colour-based detection, dramatically reducing false positives from static scene elements.

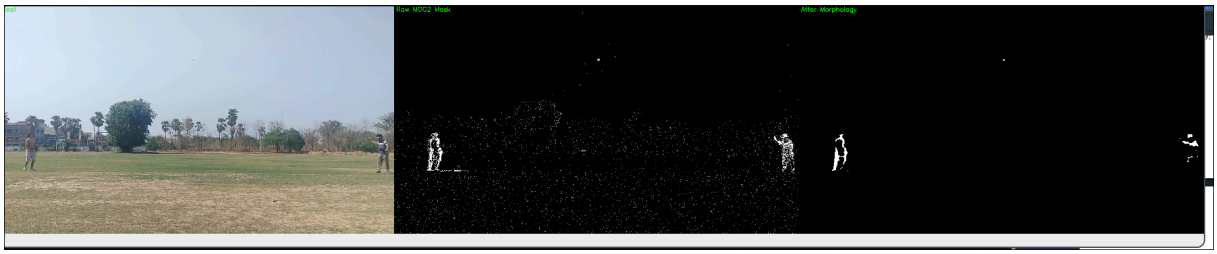


Figure 1: Left: raw video frame. Center: MOG2 foreground mask isolating the ball and player motion. Right: The video after applying morphological filtering. The ball appears as a small bright blob.

The raw foreground mask from MOG2 is rarely clean. It typically contains scattered white pixels from sensor noise, thin streaks from motion blur, and small isolated patches where the background model has not yet settled. To deal with this, two morphological operations are applied in sequence.

First, an **erosion** pass shrinks all foreground regions by sliding a small elliptical kernel (radius 4 pixels) over the mask and keeping only pixels where the kernel fits entirely within a foreground region. This wipes out isolated specks and thin lines that are too small to be the ball.

After erosion, some portions of the actual ball blob may have been trimmed away. A **dilation** pass (with a slightly larger kernel, radius 9 pixels) is then applied to grow the remaining regions back outward. The net effect is that real blobs the ball, the racket, the player arm are restored to roughly their original size, while the noise that was removed by erosion does not come back because there was nothing left to dilate.

3.2 Step 2: HSV Colour Thresholding

The foreground mask is further refined by converting the frame from BGR to HSV colour space and applying per-colour range filters. HSV separates **Hue** (colour identity) from **Saturation** and **Value** (illumination), making the filter robust to shadows and varying court lighting.

The detector supports a configurable set of target colours (white, orange, yellow, green, red, brown, black, blue). When running in `--color any` mode, the colour filter is bypassed and only shape information is used.

3.3 Step 3: Circularity Filtering and Blob Selection

After morphological cleanup (erosion to remove noise, dilation to fill gaps), contours are extracted from the combined mask. For each contour, a **circularity score** is computed based on the ratio of its area to the square of its perimeter: a perfect circle scores 1.0. Only blobs scoring above 0.6 are considered, and a strict area gate (1 to 300 px^2) rejects large non-ball objects such as player arms or score boards.

The centroid of the highest-scoring circular blob in each frame is recorded as the ball detection for that frame.

3.4 Step 4: Scale Calibration and Coordinate Conversion

Since camera intrinsics are generally unknown, a **manual scale calibration** is performed once per video: the user clicks two points of known real-world separation (e.g., the 1.75 m length of a tennis racket). The pixel distance between the clicks divided by the known real-world distance gives a scale factor s in px/m.

Pixel coordinates $(x_{\text{px}}, y_{\text{px}})$ are then converted to metric coordinates. The y-axis is flipped since pixel row indices increase downward while physical height increases upward:

$$x_m = \frac{x_{\text{px}} - x_{\min}}{s}, \quad y_m = \frac{H_{\text{frame}} - y_{\text{px}}}{s} \tag{7}$$

Figure 2: The calibration window shows the first video frame. The user clicks two points at a known real-world distance; the computed scale is printed and used for all subsequent conversions.

4 Data Filtering

Even after the three-stage detector, the raw trajectory contains a number of false detections: typically player arms, racket edges, or background clutter that briefly passes the circularity filter. Two filtering strategies are implemented.

4.1 Interactive Lasso Tool

An OpenCV window plots all detected positions on a **Time vs Vertical Position** (t-y) axes (rather than x-y). This choice is deliberate: player limbs that cross the scene at the wrong time appear as disconnected, non-parabolic clusters when viewed as y vs t, making them visually easy to identify and remove.

The user can:

- **Left-click** near a point to toggle it on/off individually.
- **Left-click-and-drag** to draw a free-form polygon (lasso); all enclosed points are immediately removed.

After editing, only the toggled-on points are passed to the model training step.

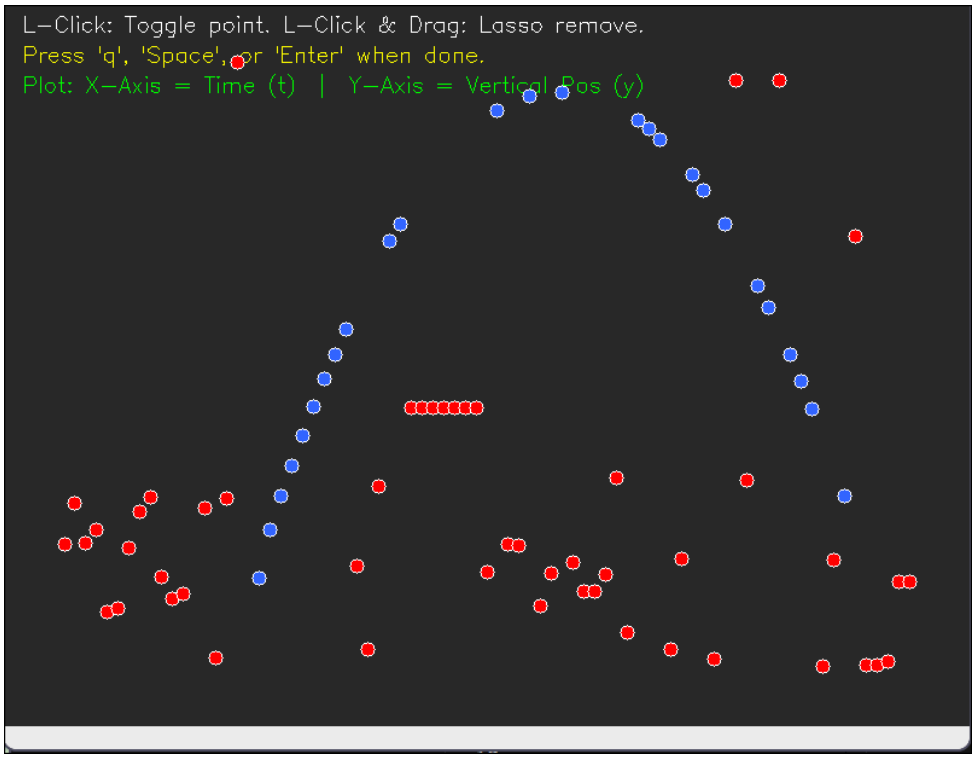


Figure 3: The lasso tool displays all detected positions in t-y space. Blue points are kept; red points are discarded. The user draws a polygon around noise clusters to remove them in bulk.

4.2 RANSAC-Based Automatic Filtering

For automated runs (without user interaction), a **RANSAC** (Random Sample Consensus) algorithm identifies the dominant downward parabolic arc in the $y(t)$ data and discards everything outside it. The algorithm works as follows:

1. Randomly sample 3 detection points.
2. Fit a quadratic $y = at^2 + bt + c$ through them.
3. **Reject** the fit if $a \geq 0$ (an upward parabola is physically impossible under gravity).
4. Count how many of all N points fall within $\tau = 0.5$ m of this parabola: these are the **inliers**.
5. Repeat 1500 times; keep the model with the most inliers.

The physics constraint $a < 0$ is the key innovation: it ensures the algorithm only accepts trajectories consistent with a gravitational arc, making it robust against noise bursts and rising artefacts.

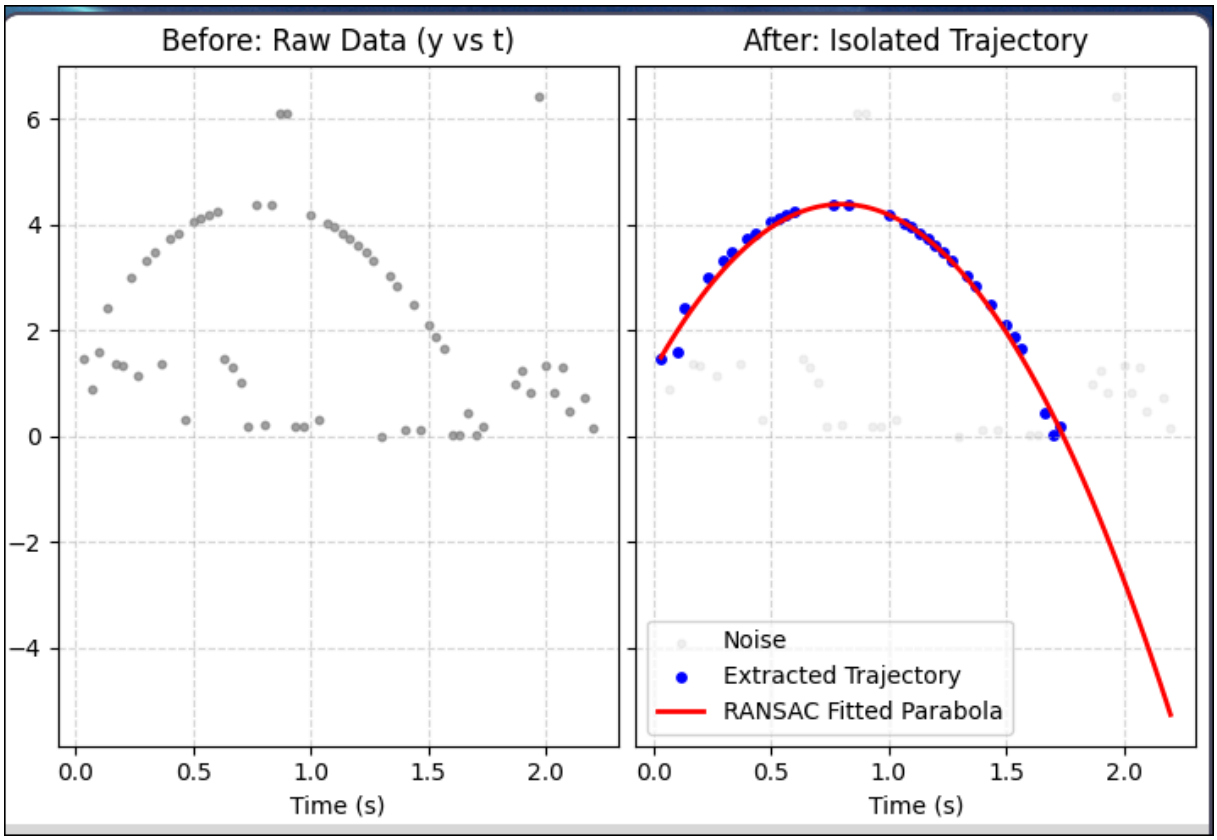


Figure 4: Left: raw y vs t scatter with heavy noise. Right: RANSAC inliers (blue) with the fitted downward parabola (red). Grey points are rejected noise.

5 Trajectory Prediction Models

Three models are trained and compared on every experiment run.

5.1 Physics-Informed Neural Network (PINN)

5.1.1 Architecture

The PINN is a 4-hidden-layer fully-connected network with 32 tanh neurons per layer, mapping normalised time $\tilde{t} \in [0, 1]$ to normalised 2D position $[\tilde{x}, \tilde{y}]$. Two additional **learnable scalar parameters**: $\log C_D$ and $\log \omega$: are trained alongside the network weights, representing the aerodynamic drag coefficient and spin rate respectively. Positivity is guaranteed via the exponential: $C_D = e^{\log C_D}$.

All inputs and outputs are linearly normalised to the unit interval to improve numerical conditioning.

5.1.2 Loss Function

The total loss balances two objectives:

Data loss: the network must fit the observed (t_i, x_i, y_i) detections:

$$\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_{i=1}^{N} \Big[\big(\tilde{x}_\theta(\tilde{t}_i) - \tilde{x}_i\big)^2 + \big(\tilde{y}_\theta(\tilde{t}_i) - \tilde{y}_i\big)^2 \Big] \tag{8}$$

Physics loss: the network's output must satisfy the aerodynamic ODE residuals at 300 additional **collocation points** spread across the trajectory window. Exact derivatives $\dot{x}, \ddot{x}, \dot{y}, \ddot{y}$ are computed analytically using PyTorch's `autograd` engine, ensuring no approximation error:

$$\mathcal{L}_{\text{phys}} = \frac{1}{N_c} \sum_{j=1}^{N_c} \left(R_x^2(\hat{t}_j) + R_y^2(\hat{t}_j) \right) \tag{9}$$

where R_x and R_y are the ODE residuals from Section 2.3. The combined loss is:

$$\mathcal{L} = (1-\beta)\mathcal{L}_{\text{data}} + \beta \mathcal{L}_{\text{phys}}, \quad \beta = 10^{-3} \tag{10}$$

The small β ensures the network first fits the measured data precisely, while the physics residual acts as a soft regulariser preventing physically impossible trajectory shapes.

5.1.3 Optimisation

The network is trained with the Adam optimiser (initial learning rate $\eta_0 = 5 \times 10^{-3}$, 5000 epochs). To prevent oscillation in the later training stages and allow fine convergence, the learning rate is annealed using a **cosine schedule**: the rate smoothly decays following the shape of a cosine curve from η_0 down to $\eta_{\min} = 10^{-5}$:

$$\eta_k = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos\left(\frac{\pi k}{K_{\max}} \right) \right) \tag{11}$$

This avoids the abrupt drops of step-decay schedules and empirically yields more stable convergence for physics-constrained problems.

5.1.4 Output: Equations of Motion and Physical Parameters

After training, the PINN reports:

- **Initial velocity** (v_{x0}, v_{y0}) extracted via autograd at $t = 0$, giving the linearised equations of motion $x(t) \approx x_0 + v_{x0}t$ and $y(t) \approx y_0 + v_{y0}t - \frac{1}{2}gt^2$.
- **Drag coefficient** $C_D = e^{\log C_D}$: a physically interpretable measure of aerodynamic resistance.
- **Spin rate** $\omega = e^{\log \omega}$ in rad/s, converted to revolutions per second (rps) for reporting.
- **Launch speed** $V_0 = \sqrt{v_{x0}^2 + v_{y0}^2}$ in m/s.

5.2 Kalman Filter

The Kalman Filter [5] is a well-established recursive Bayesian estimator that produces minimum-variance state estimates by combining a dynamic motion model with noisy measurements. It is widely used in object tracking because it naturally handles missing detections and measurement noise without storing the full observation history.

In this pipeline the Kalman Filter serves as a **classical tracking baseline**. The 6-dimensional state vector captures position, velocity, and acceleration:

$$\boldsymbol{s} = [x, y, \dot{x}, \dot{y}, \ddot{x}, \ddot{y}]^T \tag{12}$$

5.2.1 Prediction Step

At each frame the filter first **predicts** the next state using a constant-acceleration kinematic model. The state transition matrix $\boldsymbol{F}$ propagates position, velocity, and acceleration forward by Δt:

$$\boldsymbol{s}_{k|k-1} = \boldsymbol{F}\boldsymbol{s}_{k-1|k-1}, \quad \boldsymbol{P}_{k|k-1} = \boldsymbol{F}\boldsymbol{P}_{k-1|k-1}\boldsymbol{F}^T + \boldsymbol{Q} \tag{13}$$

where $\boldsymbol{P}$ is the state covariance matrix and $\boldsymbol{Q} = \sigma_q^2 \boldsymbol{I}_6$ ($\sigma_q = 1.0$) is the process noise covariance, accounting for model imperfections such as the true aerodynamic deceleration that the constant-acceleration assumption ignores.

5.2.2 Update Step

When a detection $\boldsymbol{z}_k = [x_k, y_k]^T$ is available, the filter **corrects** the prediction using the Kalman gain $\boldsymbol{K}_k$:

$$\boldsymbol{K}_k = \boldsymbol{P}_{k|k-1}\boldsymbol{H}^T\Big(\boldsymbol{H}\boldsymbol{P}_{k|k-1}\boldsymbol{H}^T + \boldsymbol{R}\Big)^{-1} \tag{14}$$

$$\boldsymbol{s}_{k|k} = \boldsymbol{s}_{k|k-1} + \boldsymbol{K}_k\Big(\boldsymbol{z}_k - \boldsymbol{H}\boldsymbol{s}_{k|k-1}\Big) \tag{15}$$

$$\boldsymbol{P}_{k|k} = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{H})\boldsymbol{P}_{k|k-1} \tag{16}$$

Here $\boldsymbol{H} \in \mathbb{R}^{2\times 6}$ is the measurement matrix that extracts (x, y) from the full state, and $\boldsymbol{R} = \sigma_r^2\boldsymbol{I}_2$ ($\sigma_r = 5.0$ m) is the measurement noise covariance. The Kalman gain automatically weights the prediction and measurement according to their relative uncertainties: a large $\boldsymbol{R}$ causes the filter to trust its own motion model more; a large $\boldsymbol{Q}$ causes it to trust the measurement more.

5.2.3 Initialisation and Extrapolation

The filter is initialised with $\boldsymbol{s}_0 = [x_0, y_0, 0, 0, 0, 0]^T$ and a diagonal covariance $\boldsymbol{P}_0 = 100\boldsymbol{I}_6$ to reflect high initial uncertainty. After processing all observed frames, future positions are extrapolated by repeated application of $\boldsymbol{F}$ without any measurement update, relying entirely on the estimated velocity and acceleration at the last detected frame.

The key limitation of the Kalman Filter here is that its constant-acceleration model does not encode the direction of gravity or drag. It therefore tends to drift upward in regions of high vertical acceleration change and is outperformed by the PINN once the trajectory curves significantly. See [2] for a comparative analysis of Kalman-based trackers on fast-moving objects.

5.3 Parabolic Regression

The classical vacuum-physics baseline fits a spatial parabola $y = ax^2 + bx + c$ directly to the observed (x, y) coordinate pairs using nonlinear least squares (Levenberg–Marquardt). Horizontal position at query times is obtained by linear interpolation from the first to the last detected x position (preserving the direction of travel). This model makes no attempt to account for drag or spin and serves as the lower-accuracy reference.

5.4 Measured vs PINN Velocity

The velocity analysis plots show two distinct velocity estimates plotted together: **Measured** and **PINN**: which are computed by fundamentally different methods and are worth distinguishing clearly.

Measured velocity is obtained by simple **first-order finite differences** applied directly to the cleaned position data. For consecutive detections at times t_i and $t_{\{i+1\}}$:

$$v_x^{\text{meas}(t_m)} = \frac{x_{i+1} - x_i}{t_{i+1} - t_i}, \quad v_y^{\text{meas}(t_m)} = \frac{y_{i+1} - y_i}{t_{i+1} - t_i} \tag{17}$$

where $t_m = \frac{t_i + t_{\{i+1\}}}{2}$ is the midpoint time. This is a direct, model-free estimate. Its accuracy depends entirely on how densely and regularly the ball was detected: if frames were dropped or a detection was slightly mislocated, the finite difference amplifies that positional error into a large velocity spike.

PINN velocity is computed by applying PyTorch's `autograd` engine to differentiate the trained network output analytically with respect to time:

$$v_x^{\text{PINN}(t)} = \frac{\partial \tilde{x}}{\partial \tilde{t}} \cdot \frac{x_{\text{scale}}}{t_{\text{scale}}} \tag{18}$$

Because the PINN has learned a smooth, physics-consistent trajectory: one that must simultaneously satisfy the aerodynamic ODEs: its velocity is inherently smooth and physically plausible everywhere, including at times between actual detections. The PINN velocity profile therefore represents the **best physically-constrained estimate** of the true velocity, while the measured velocity provides the **raw empirical signal** from the camera.

6 Evaluation Metrics

Each model is evaluated against the observed trajectory using four standard metrics:

Metric	**Formula**	**What it measures**
RMSE_x, RMSE_y	$\sqrt{\frac{1}{n}\sum_i \left(z_i^{\text{gt}} - z_i^{\text{pred}}\right)^2}$	Per-axis average squared error
ADE	$\frac{1}{n}\sum_i \sqrt{\Delta x_i^2 + \Delta y_i^2}$	Mean position error over whole trajectory
FDE	$\sqrt{\Delta x_n^2 + \Delta y_n^2}$	Position error at the final point only

Table 1: Evaluation metrics used to compare all three models

7 System Overview

Stage	**Method**	**Output**
Detection	MOG2 + HSV + Circularity	(t_i, x_i, y_i) in pixels
Calibration	Manual two-click scale	Scale s in px/m
Coordinate Conv.	Flip y, divide by s	(t_i, x_i, y_i) in metres
Pre-clean	IQR outlier gate	Reduced outlier set
Filtering	Lasso (interactive) / RANSAC	Clean trajectory
PINN	PyTorch + Adam + autograd	C_D, ω, V_0, trajectory
Kalman	Constant-accel. KF	Smoothed + extrapolated
Parabolic	Spatial curve fit $y(x)$	Vacuum baseline
Evaluation	RMSE, ADE, FDE	Comparison table + plots

Table 2: End-to-end CARS pipeline stages

8 Appendix: Experimental Results

Each subsection below shows the four output plots generated by the CARS pipeline for the corresponding experiment: (1) the full multi-panel report, (2) the trajectory comparison across models, (3) the velocity analysis, and (4) the error metrics bar chart.

8.1 Experiment 1

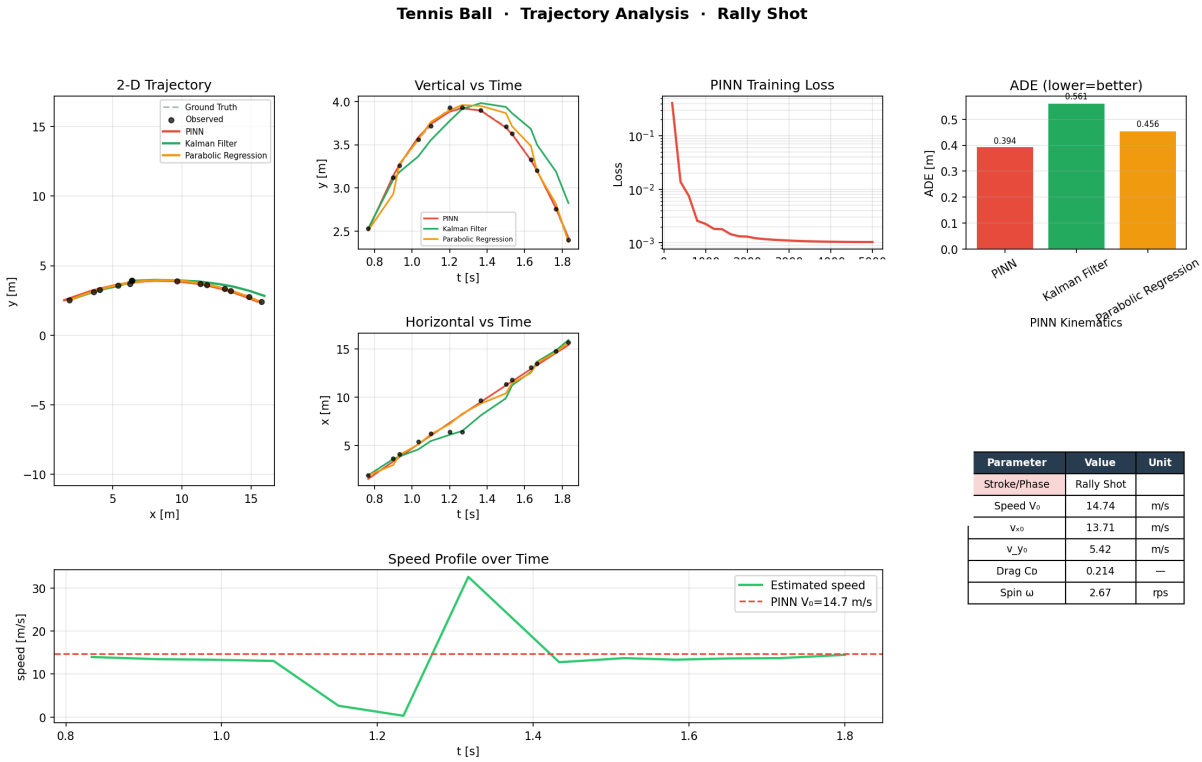


Figure 5: Full pipeline report — Experiment 1

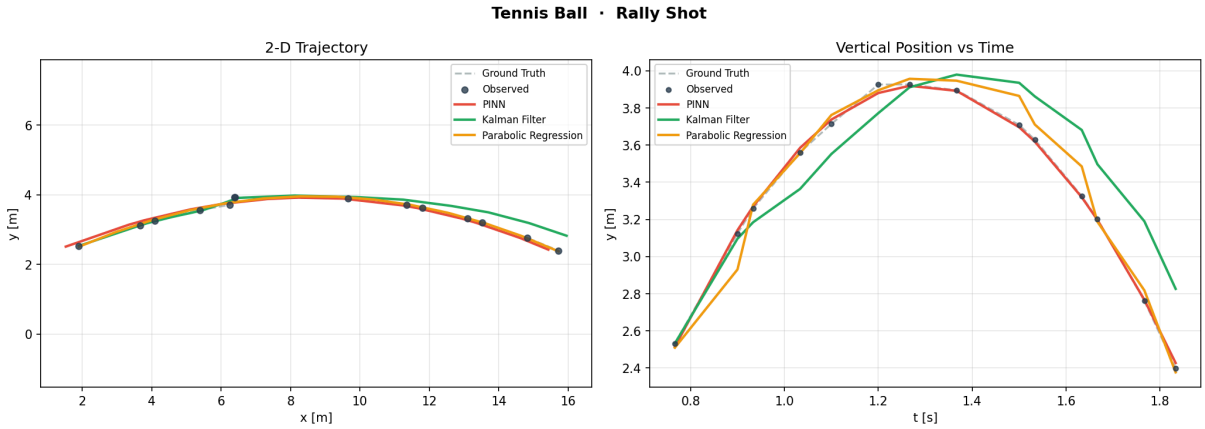


Figure 6: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 1

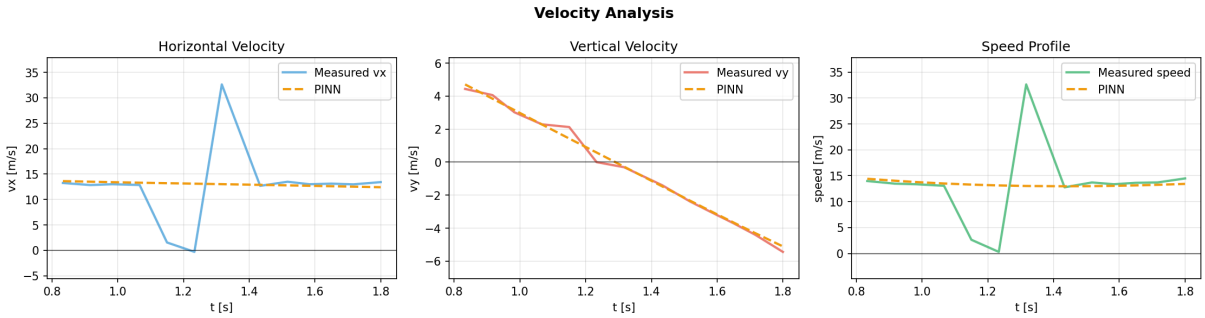
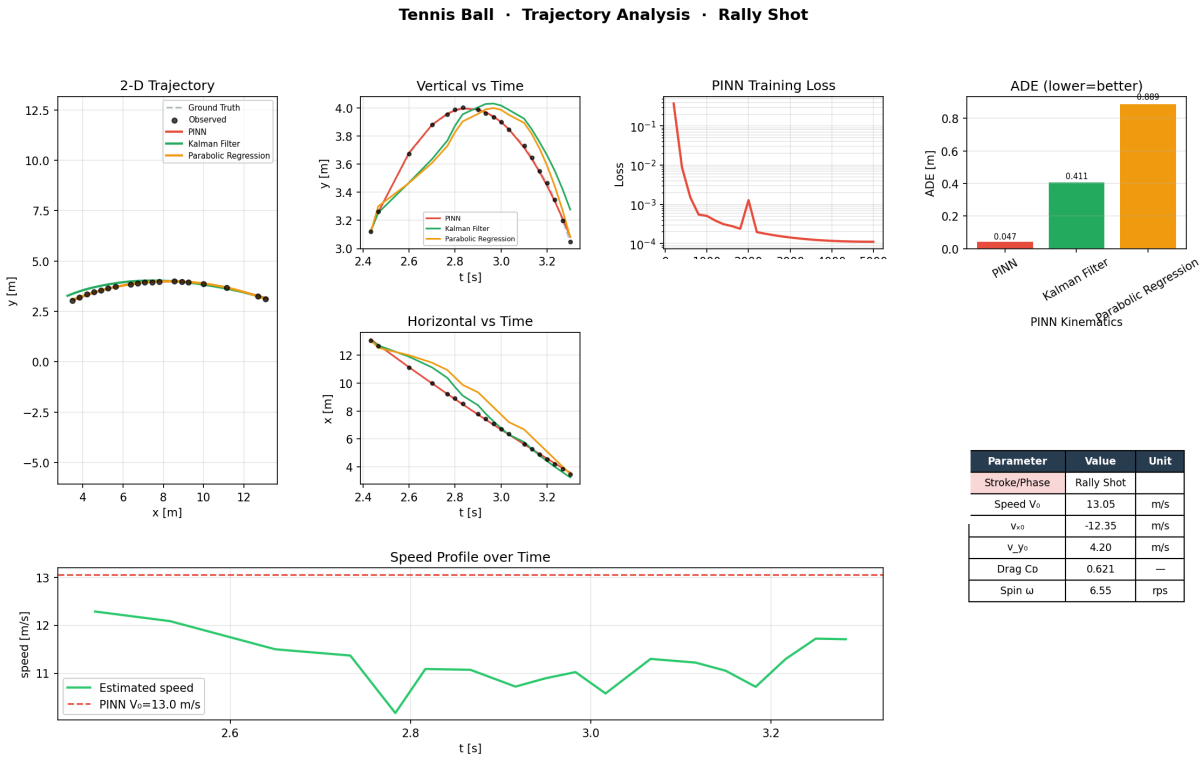


Figure 7: Velocity analysis — Experiment 1

Figure 8: Error metrics — Experiment 1

8.2 Experiment 2



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V_0	13.05	m/s
v_{x0}	-12.35	m/s
v_y_0	4.20	m/s
Drag C_D	0.621	—
Spin ω	6.55	rps

Figure 9: Full pipeline report — Experiment 2

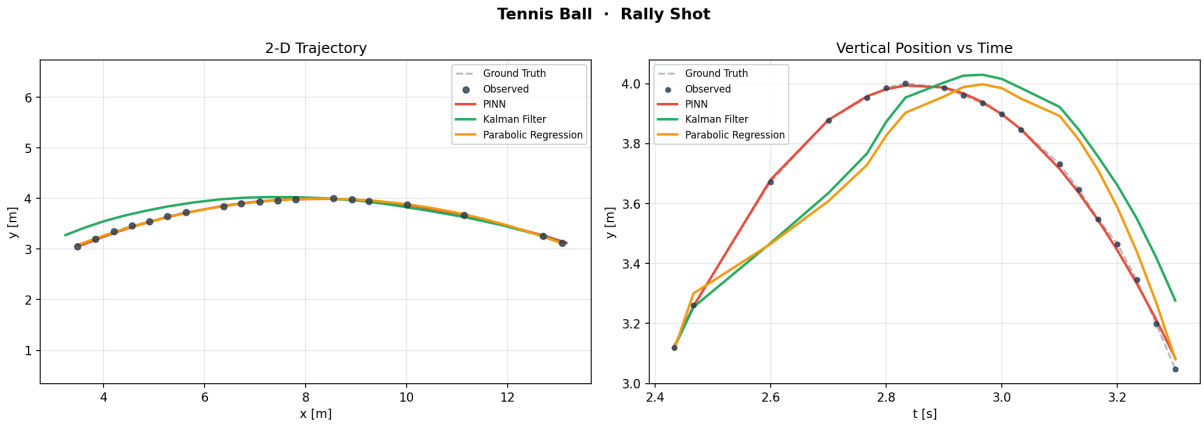


Figure 10: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 2

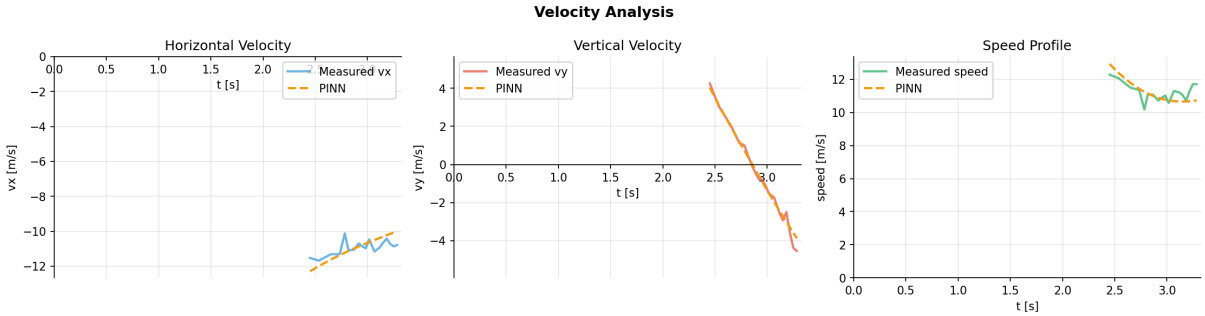


Figure 11: Velocity analysis — Experiment 2

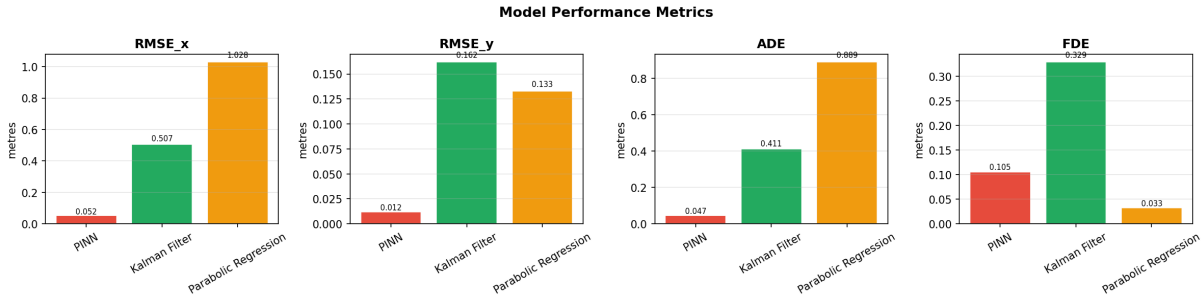
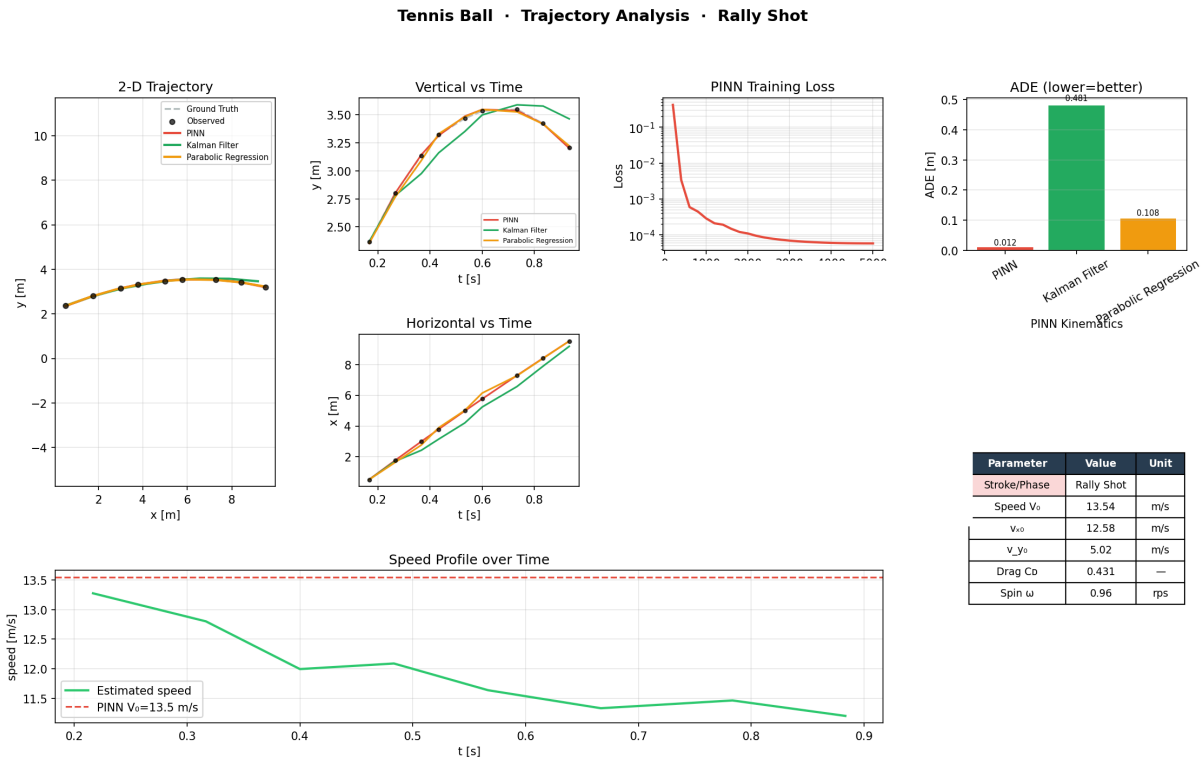


Figure 12: Error metrics — Experiment 2

8.3 Experiment 3



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V₀	13.54	m/s
vₓ₀	12.58	m/s
v_y₀	5.02	m/s
Drag Cᴅ	0.431	—
Spin ω	0.96	rps

Figure 13: Full pipeline report — Experiment 3

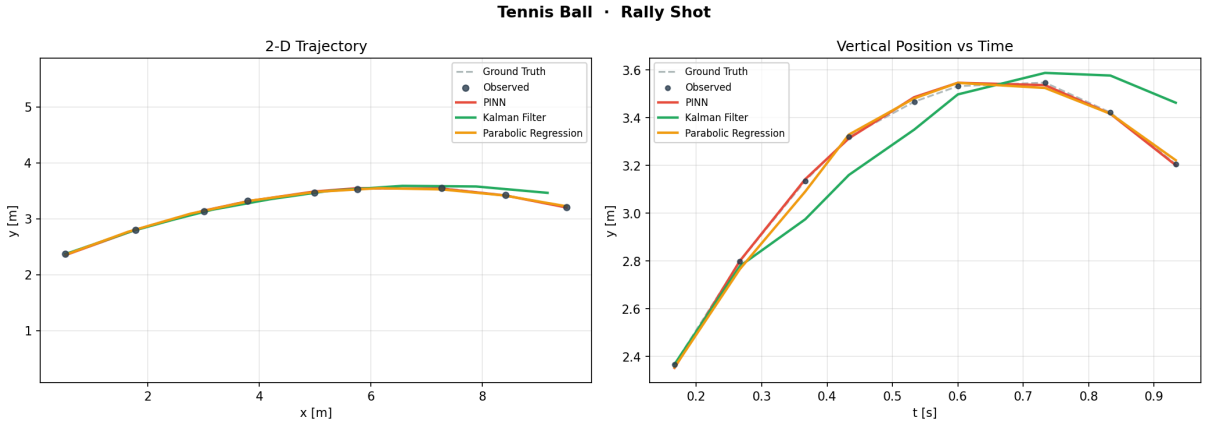


Figure 14: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 3

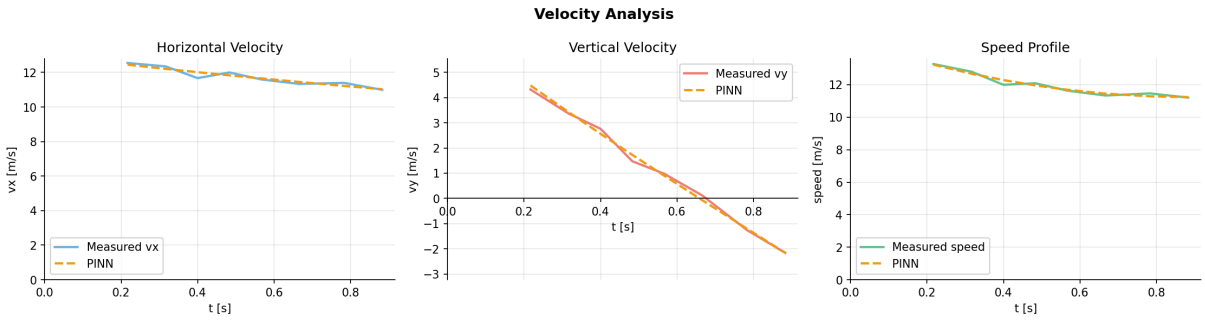


Figure 15: Velocity analysis — Experiment 3

Figure 16: Error metrics — Experiment 3

8.4 Experiment 4

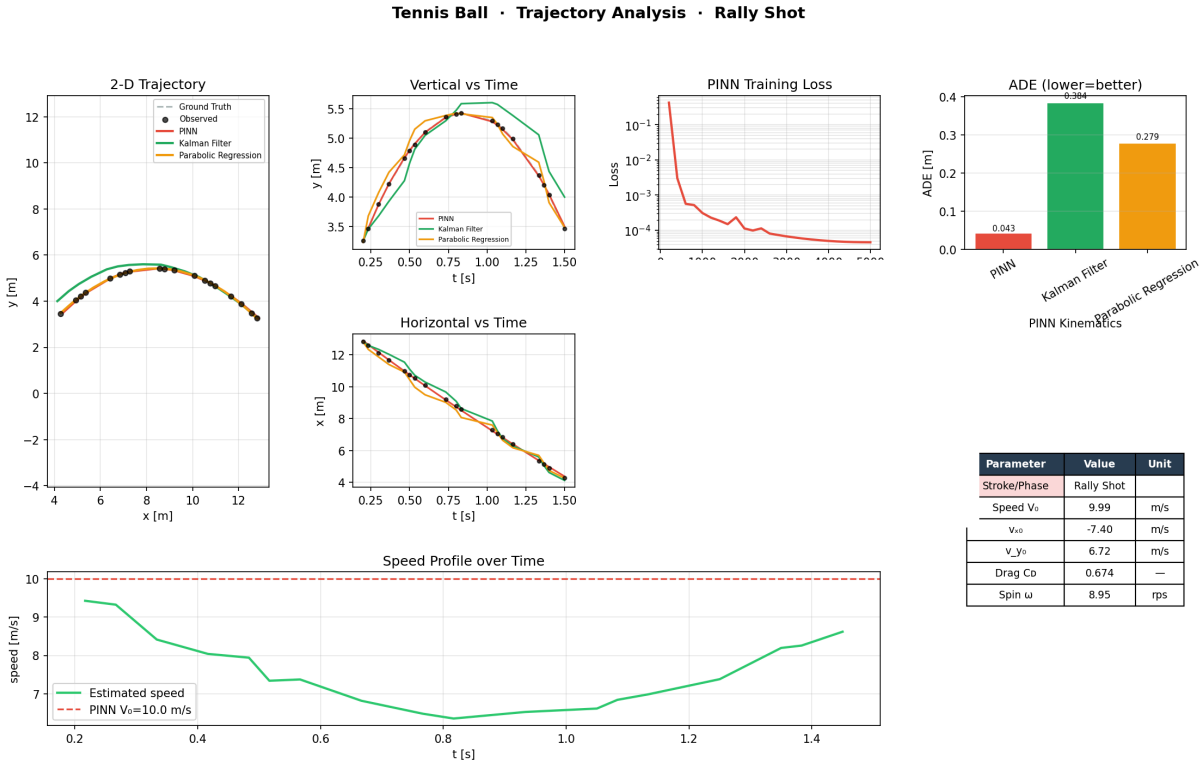


Figure 17: Full pipeline report — Experiment 4

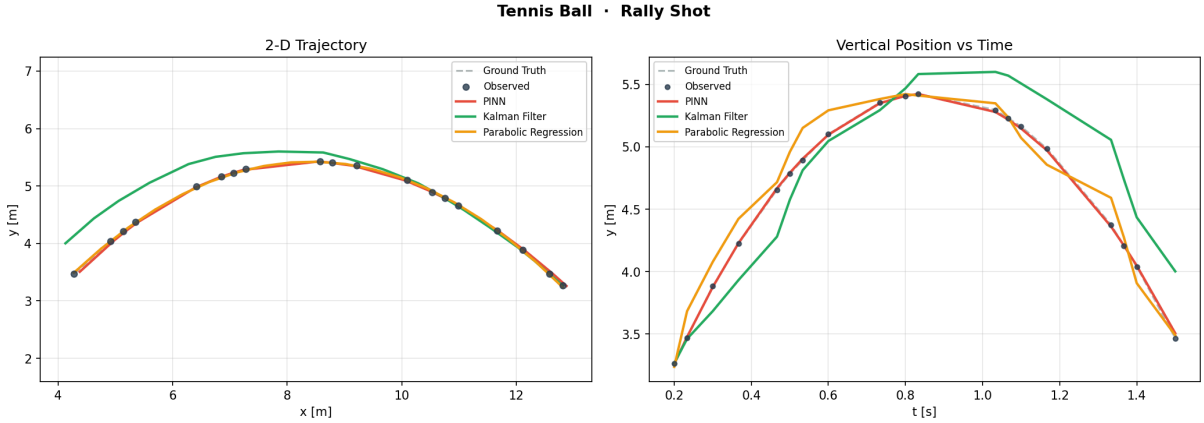


Figure 18: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 4

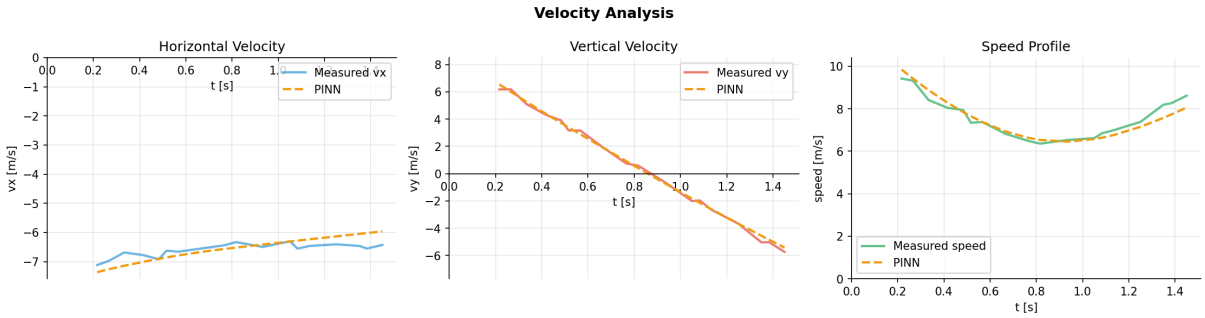
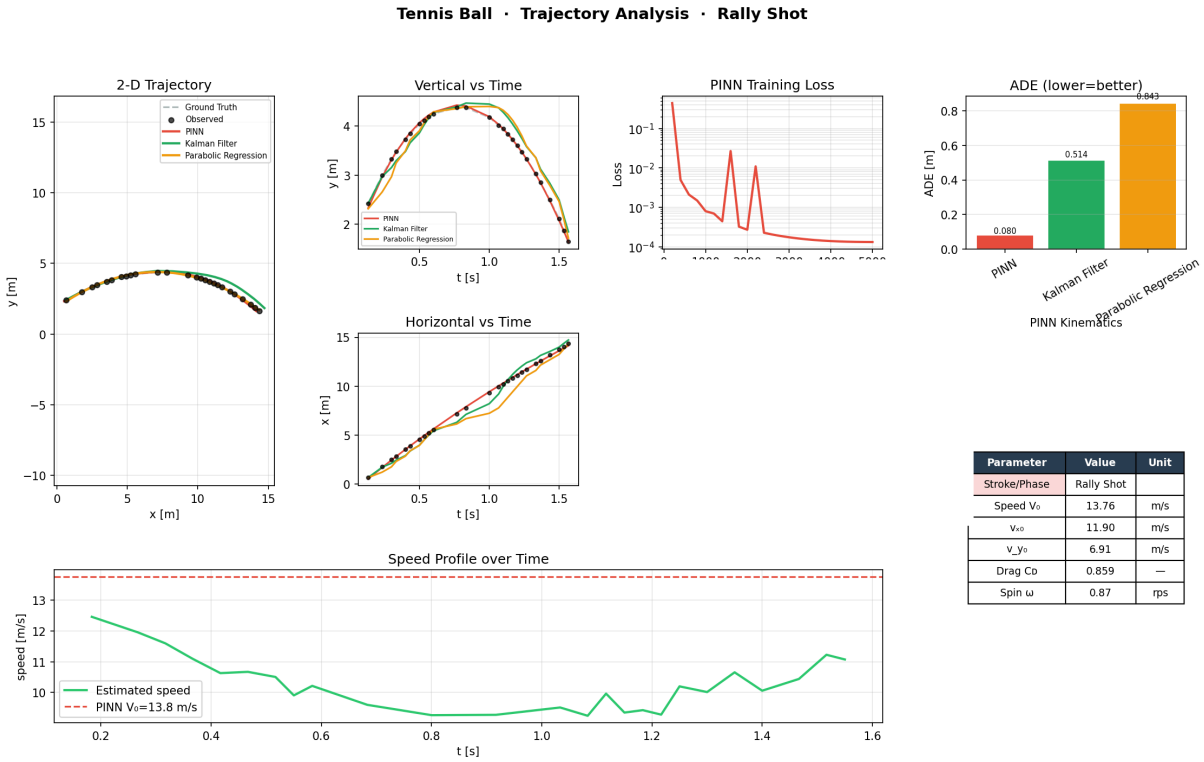


Figure 19: Velocity analysis — Experiment 4

Figure 20: Error metrics — Experiment 4

8.5 Experiment 5



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V_0	13.76	m/s
v_{x0}	11.90	m/s
v_{y0}	6.91	m/s
Drag C_D	0.859	—
Spin ω	0.87	rps

Figure 21: Full pipeline report — Experiment 5

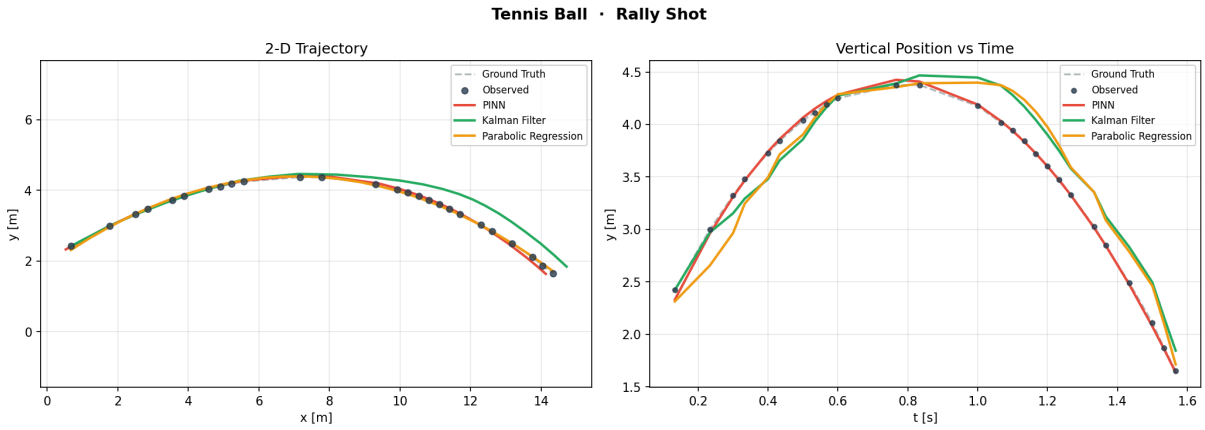


Figure 22: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 5

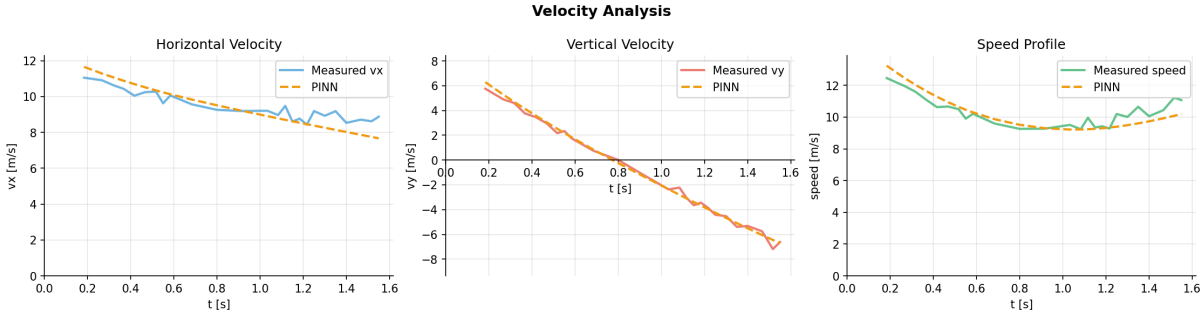
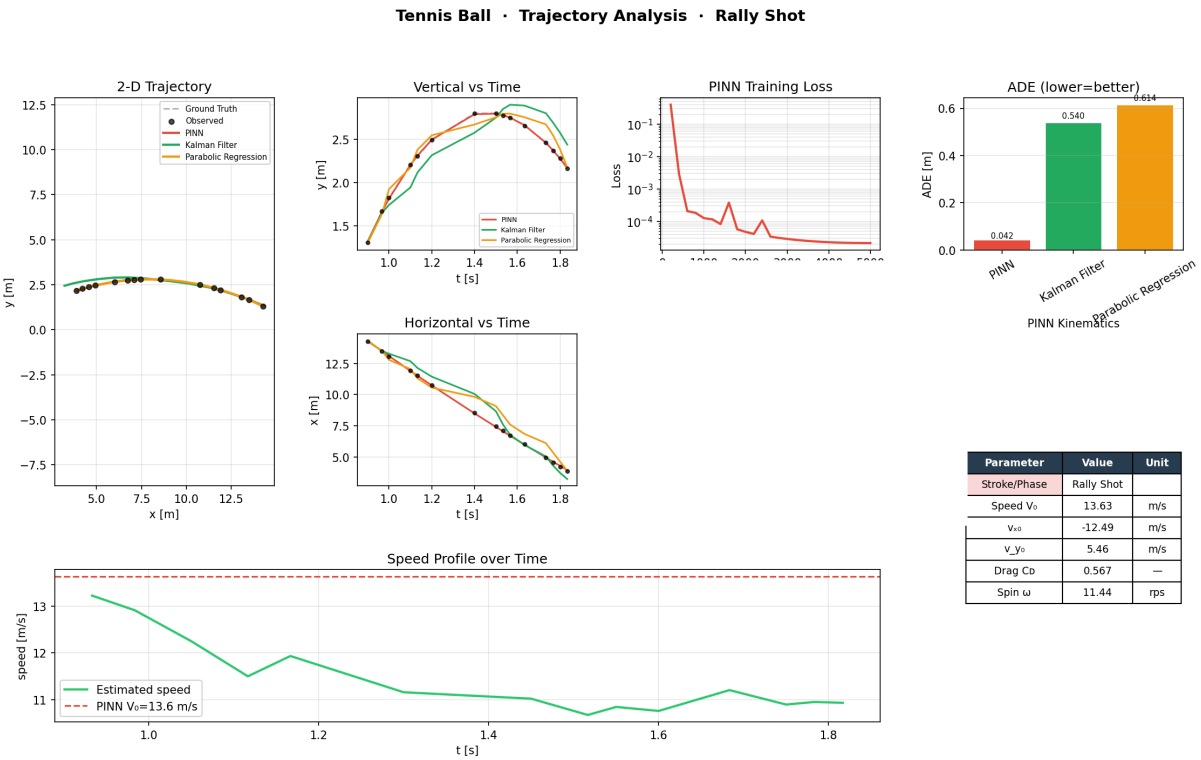


Figure 23: Velocity analysis — Experiment 5

Figure 24: Error metrics — Experiment 5

8.6 Experiment 6



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V_0	13.63	m/s
v_{x0}	-12.49	m/s
v_y_0	5.46	m/s
Drag C_D	0.567	—
Spin ω	11.44	rps

Figure 25: Full pipeline report — Experiment 6

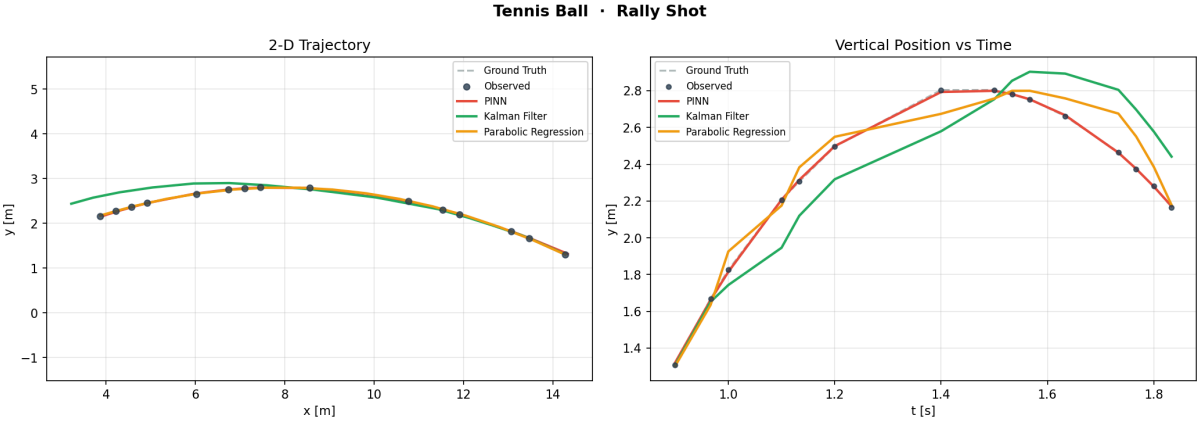


Figure 26: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 6

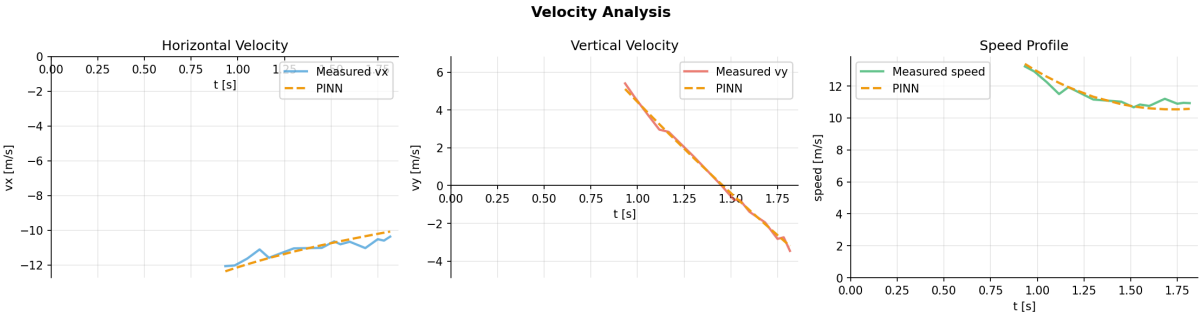


Figure 27: Velocity analysis — Experiment 6

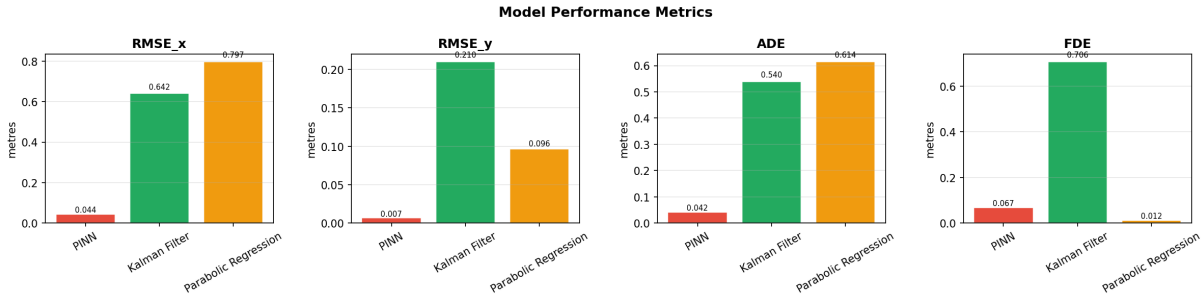
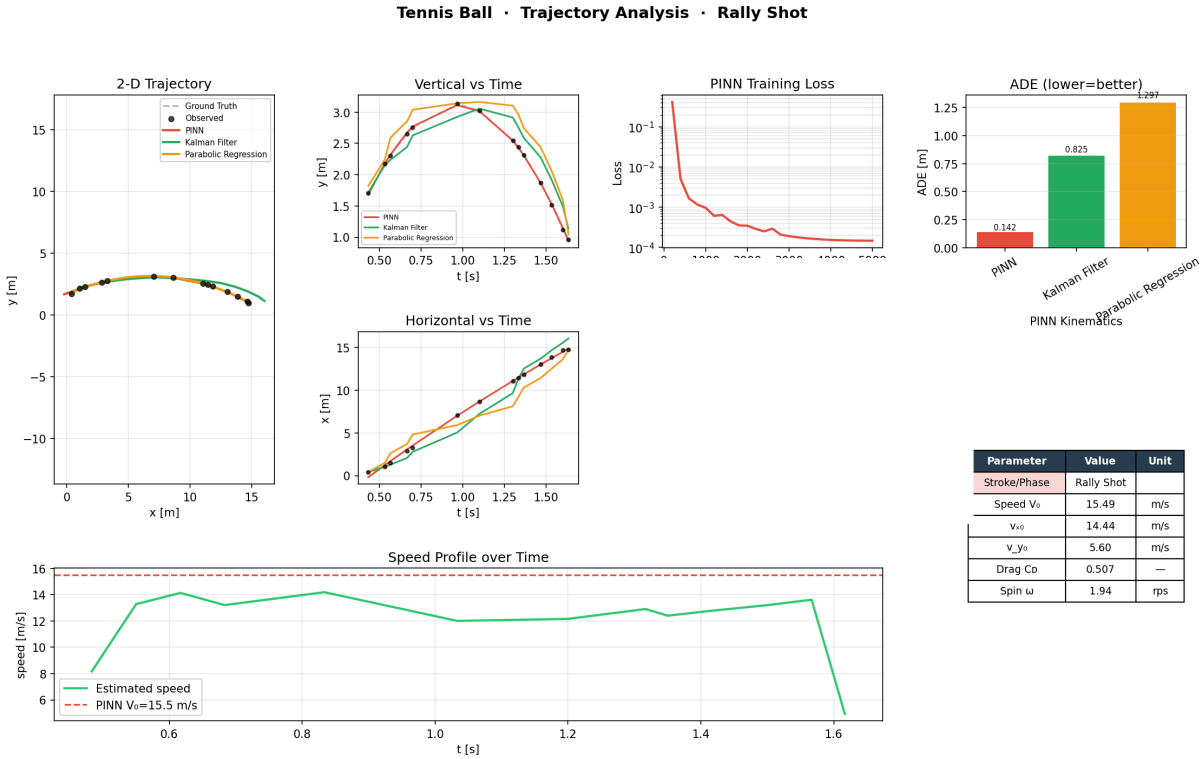


Figure 28: Error metrics — Experiment 6

8.7 Experiment 7



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V_0	15.49	m/s
v_{x0}	14.44	m/s
v_y_0	5.60	m/s
Drag Cᴅ	0.507	—
Spin ω	1.94	rps

Figure 29: Full pipeline report — Experiment 7

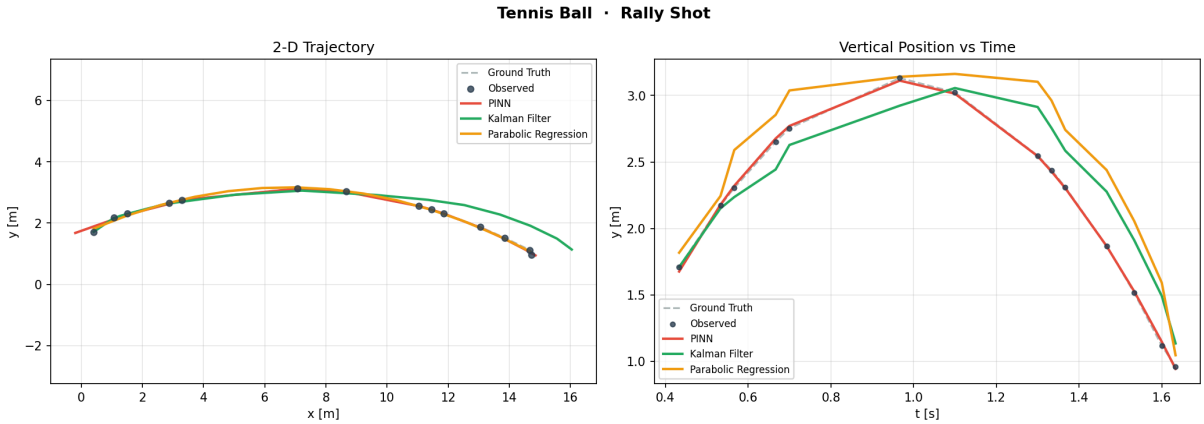


Figure 30: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 7

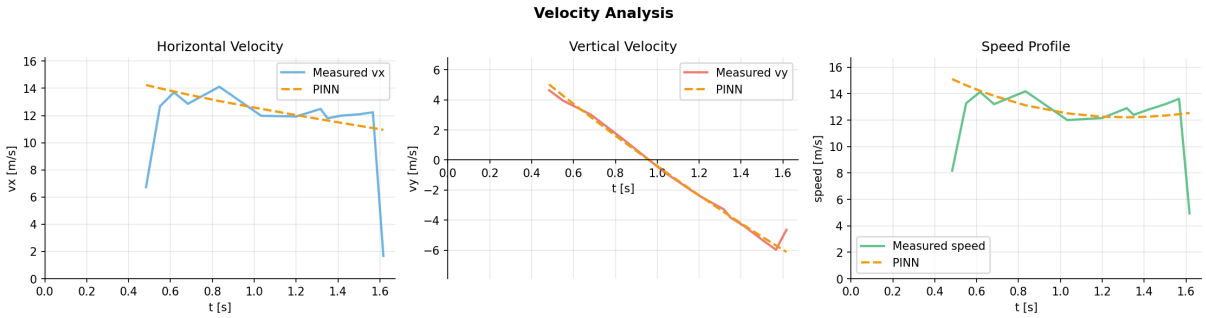


Figure 31: Velocity analysis — Experiment 7

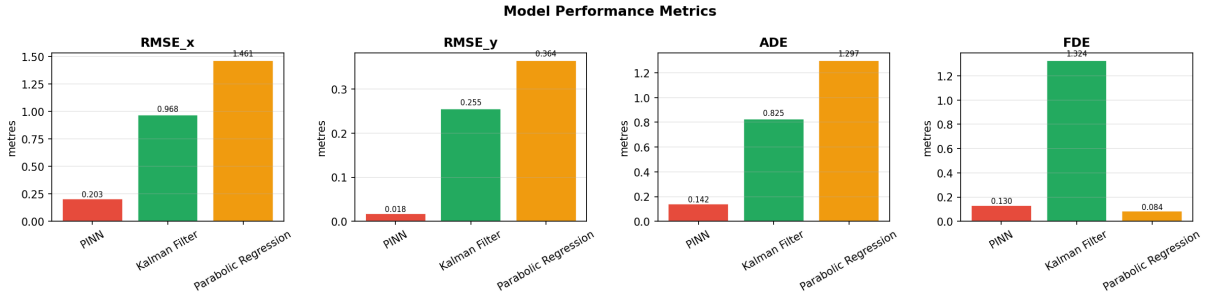
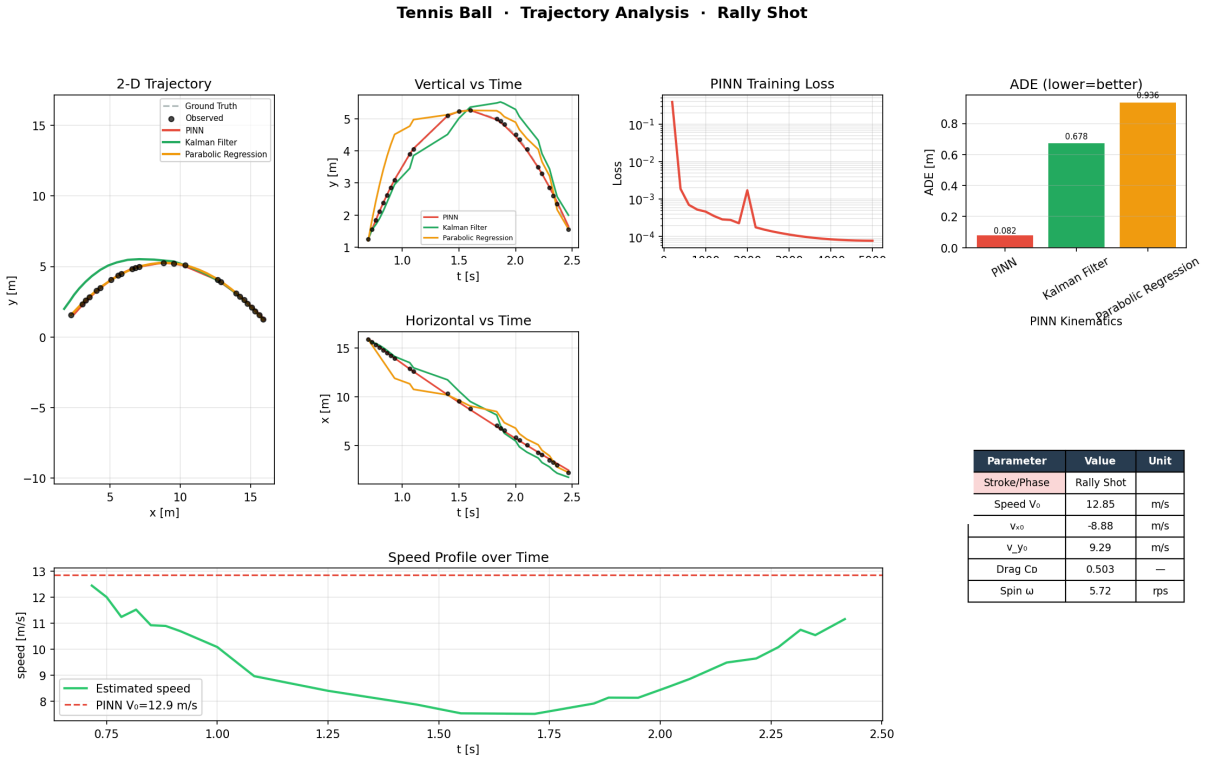


Figure 32: Error metrics — Experiment 7

8.8 Experiment 8



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V_0	12.85	m/s
v_{x0}	-8.88	m/s
v_y_0	9.29	m/s
Drag C_D	0.503	—
Spin ω	5.72	rps

Figure 33: Full pipeline report — Experiment 8

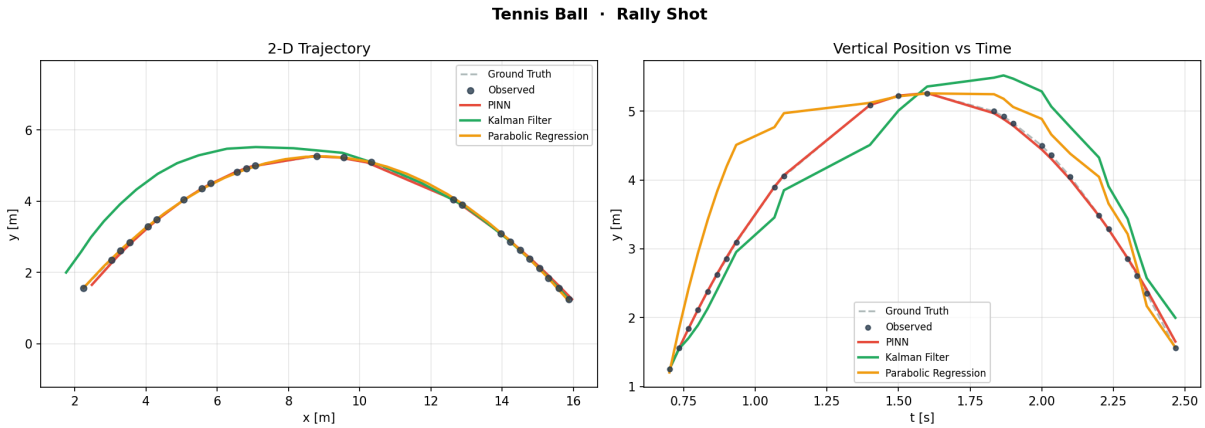


Figure 34: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 8

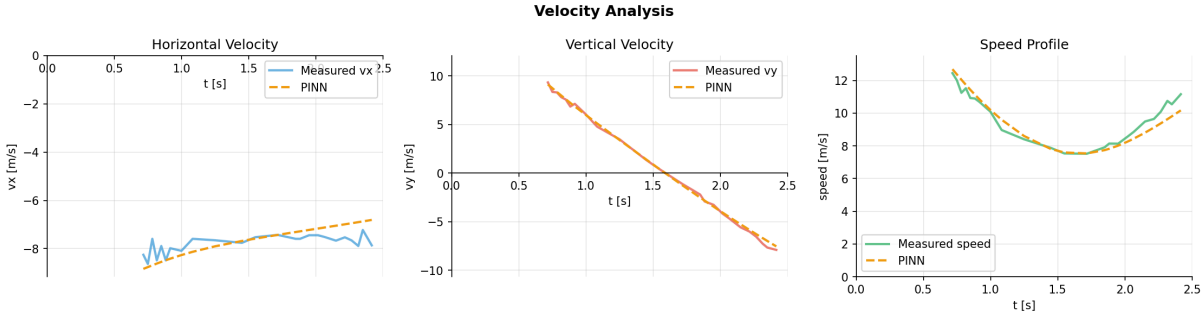


Figure 35: Velocity analysis — Experiment 8

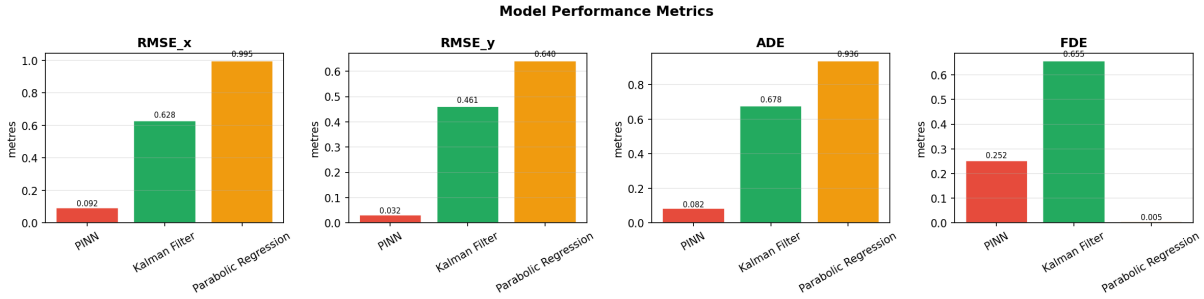


Figure 36: Error metrics — Experiment 8

8.9 Experiment 9

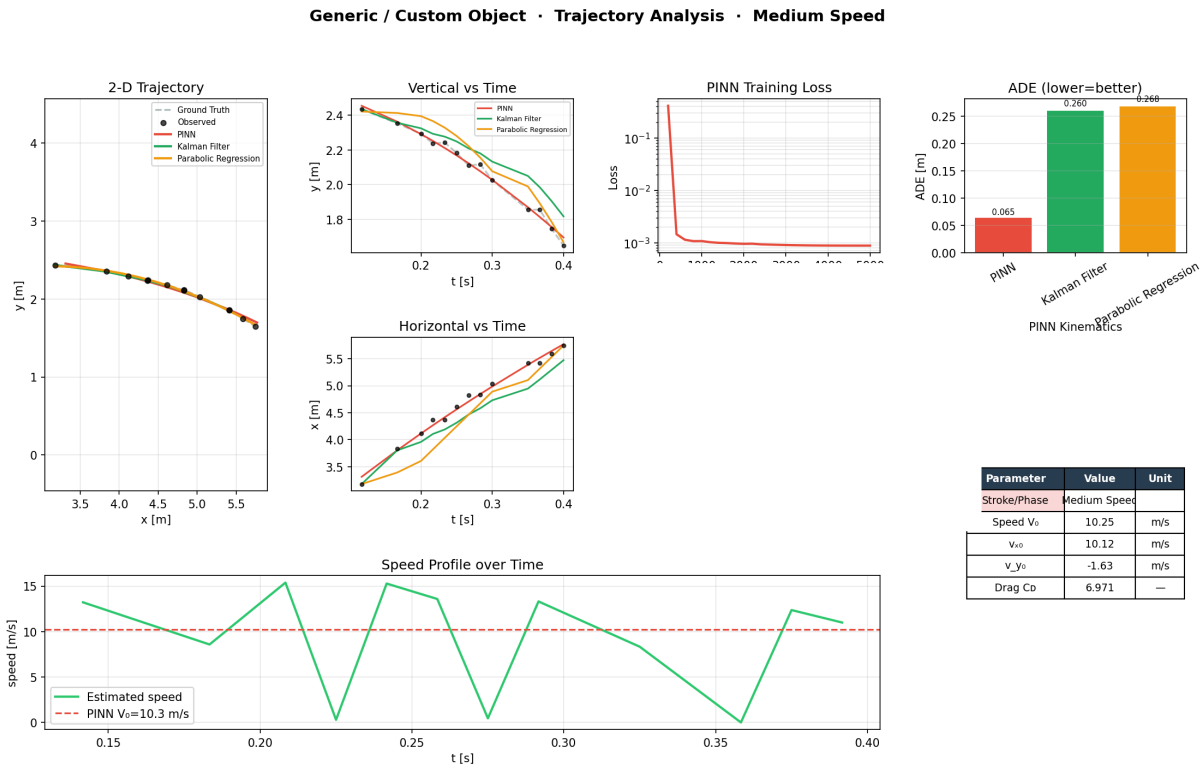


Figure 37: Full pipeline report — Experiment 9

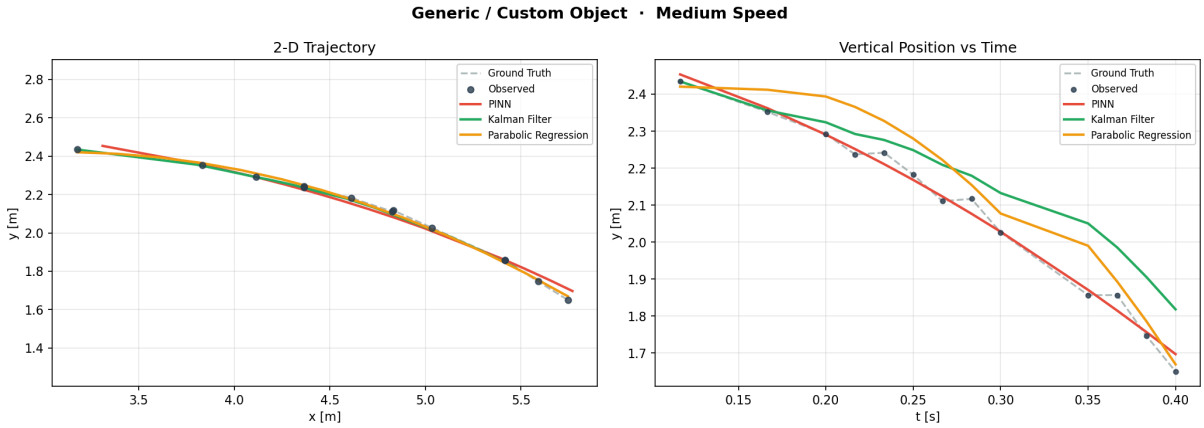


Figure 38: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 9

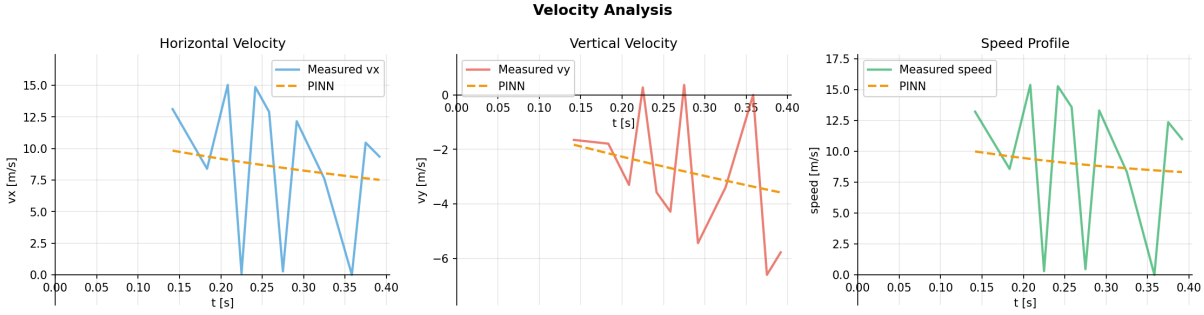
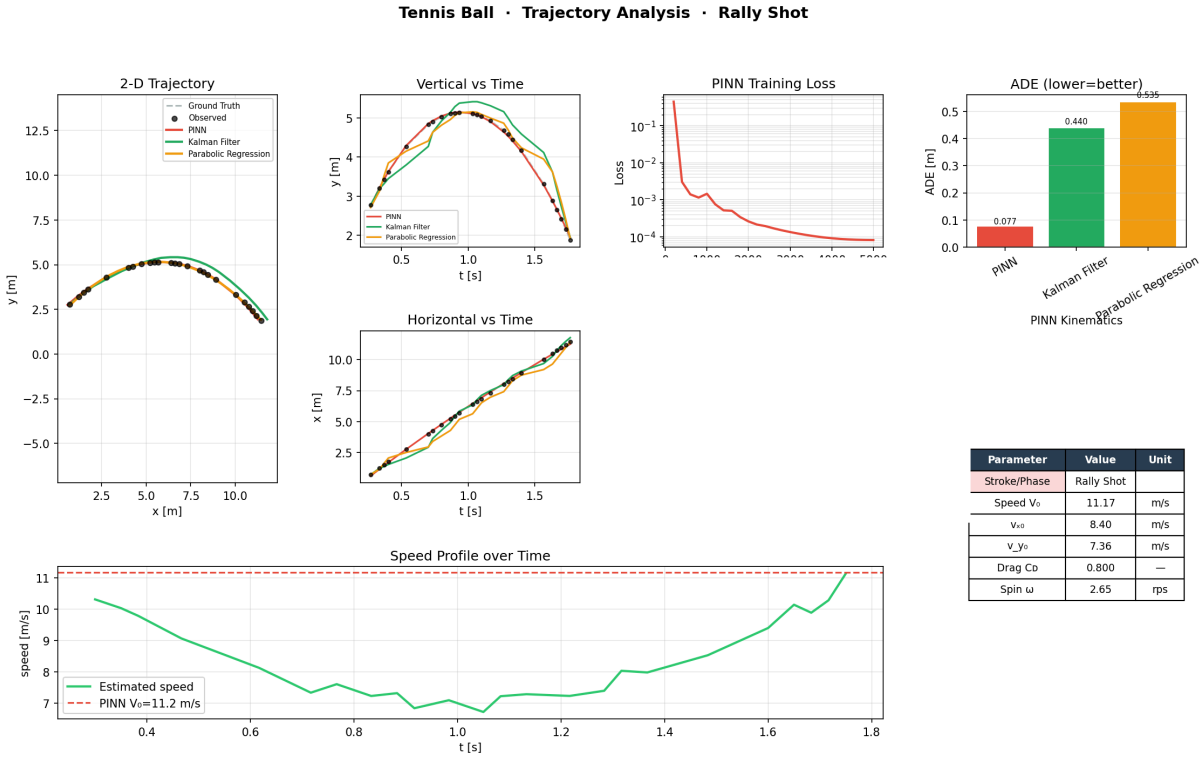


Figure 39: Velocity analysis — Experiment 9

Figure 40: Error metrics — Experiment 9

8.10 Experiment 10



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V_0	11.17	m/s
v_{x0}	8.40	m/s
v_y_0	7.36	m/s
Drag C_D	0.800	—
Spin ω	2.65	rps

Figure 41: Full pipeline report — Experiment 10

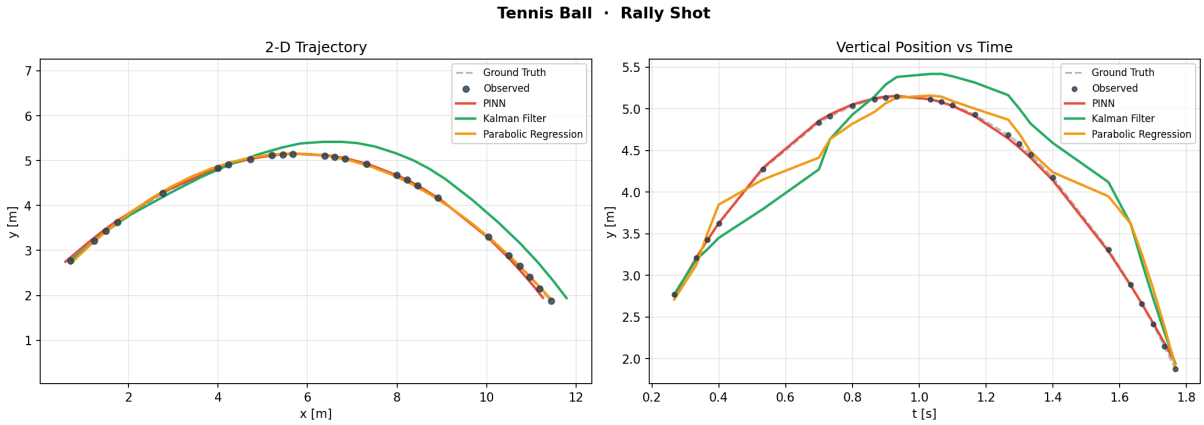


Figure 42: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 10

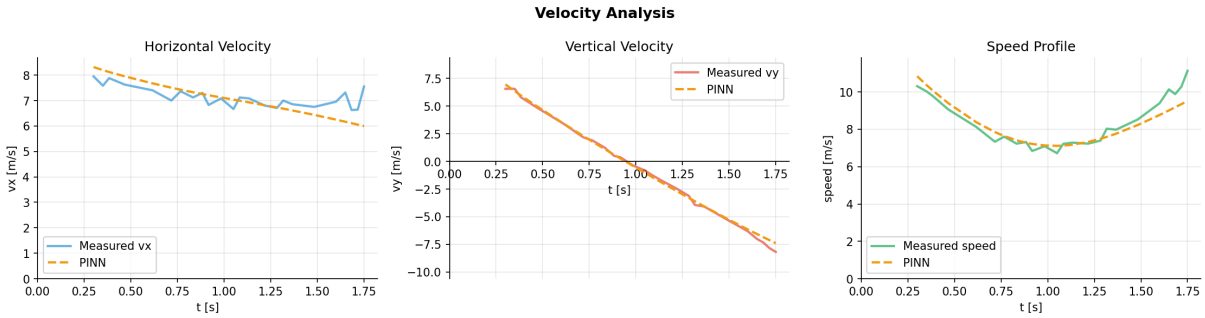


Figure 43: Velocity analysis — Experiment 10

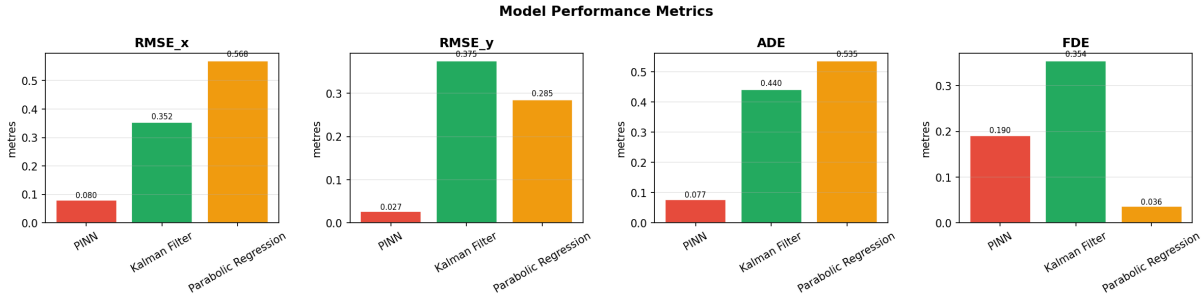
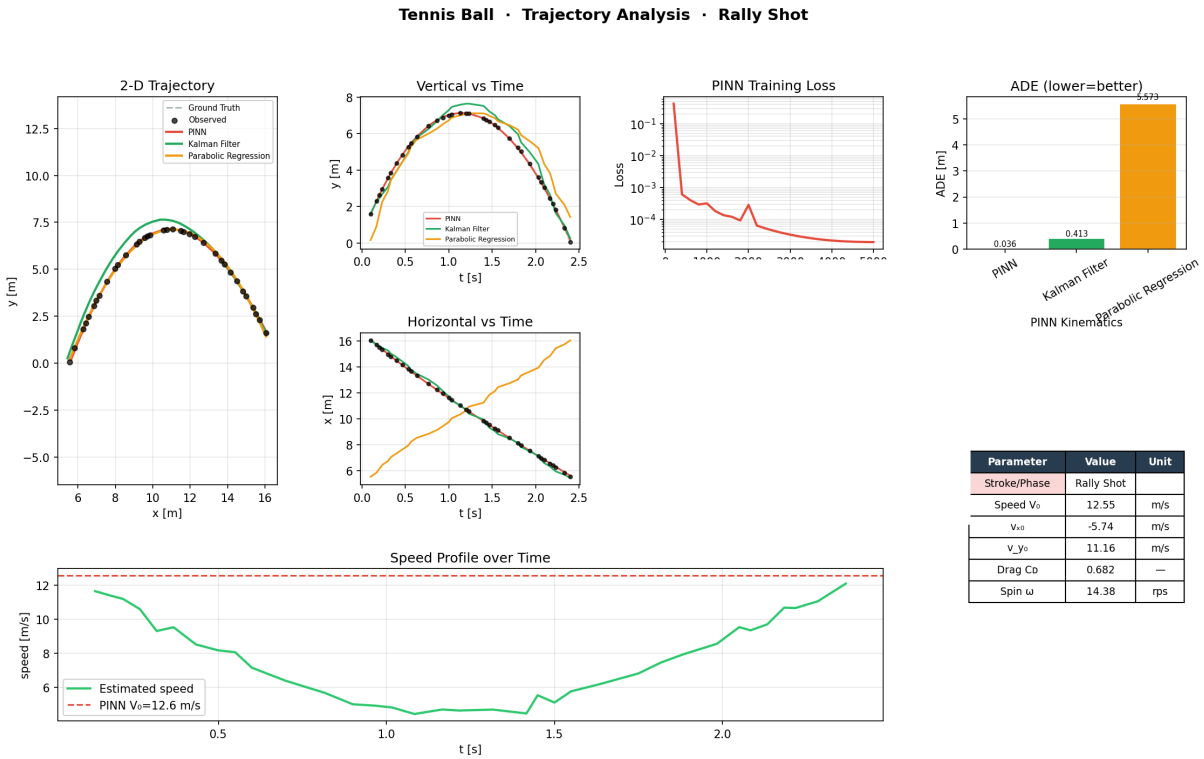


Figure 44: Error metrics — Experiment 10

8.11 Experiment 11



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V_0	12.55	m/s
v_{x0}	-5.74	m/s
v_{y0}	11.16	m/s
Drag C_D	0.682	—
Spin ω	14.38	rps

Figure 45: Full pipeline report — Experiment 11

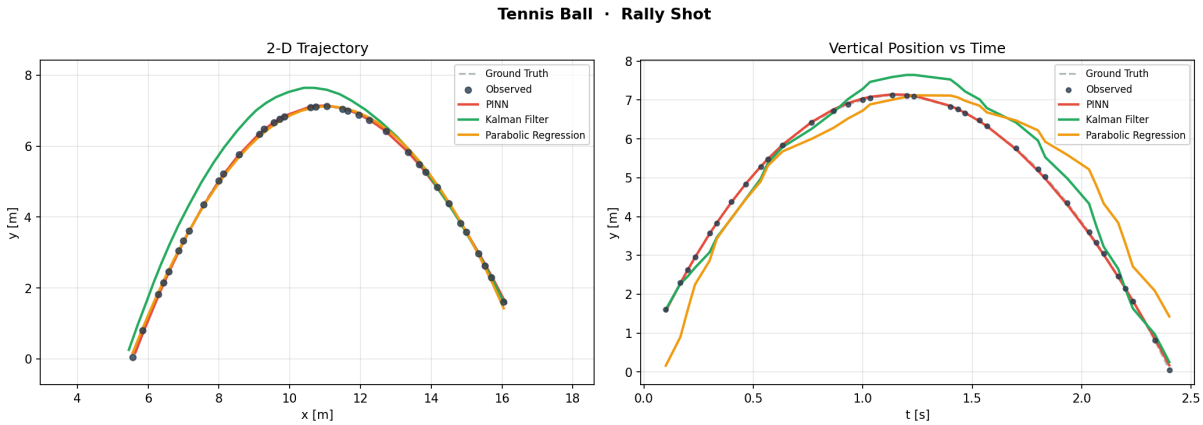


Figure 46: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 11

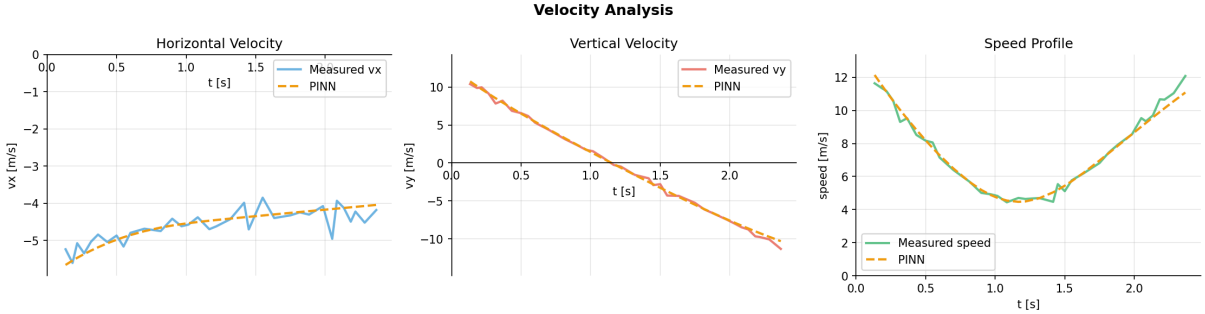
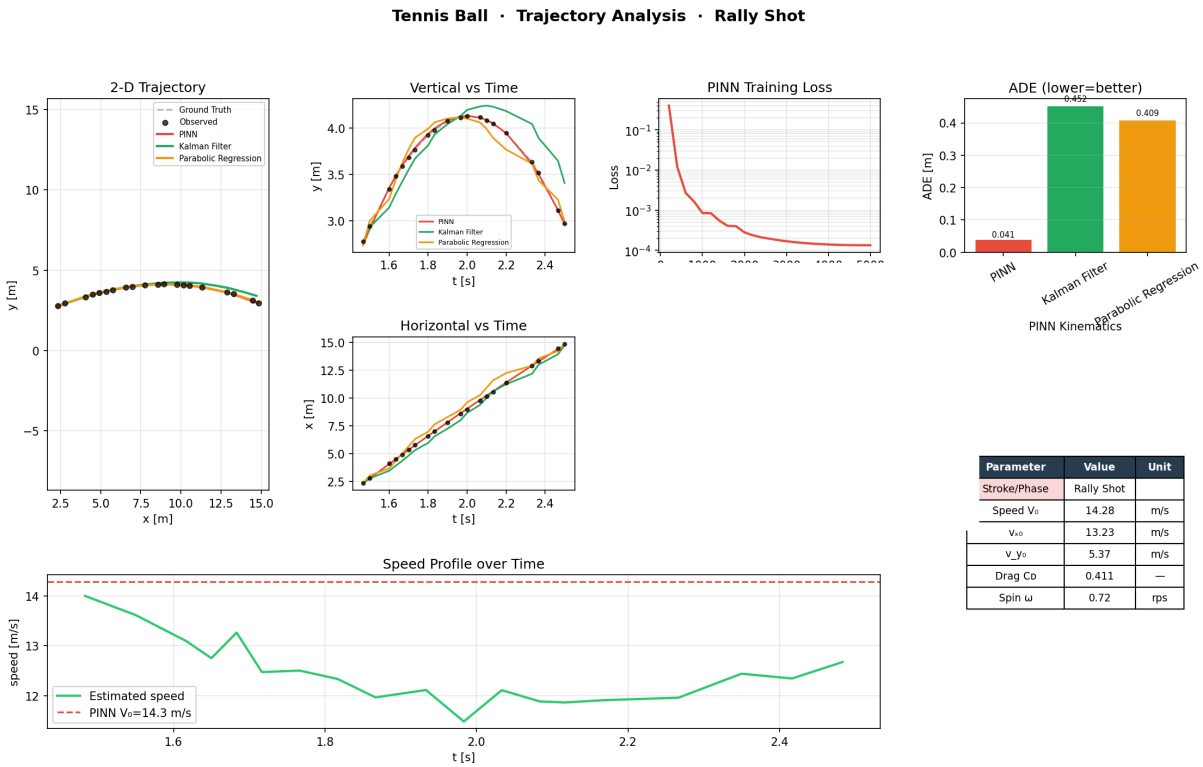


Figure 47: Velocity analysis — Experiment 11

Figure 48: Error metrics — Experiment 11

8.12 Experiment 12



Parameter	Value	Unit
Stroke/Phase	Rally Shot	
Speed V_0	14.28	m/s
v_{x0}	13.23	m/s
v_{y0}	5.37	m/s
Drag C_D	0.411	—
Spin ω	0.72	rps

Figure 49: Full pipeline report — Experiment 12

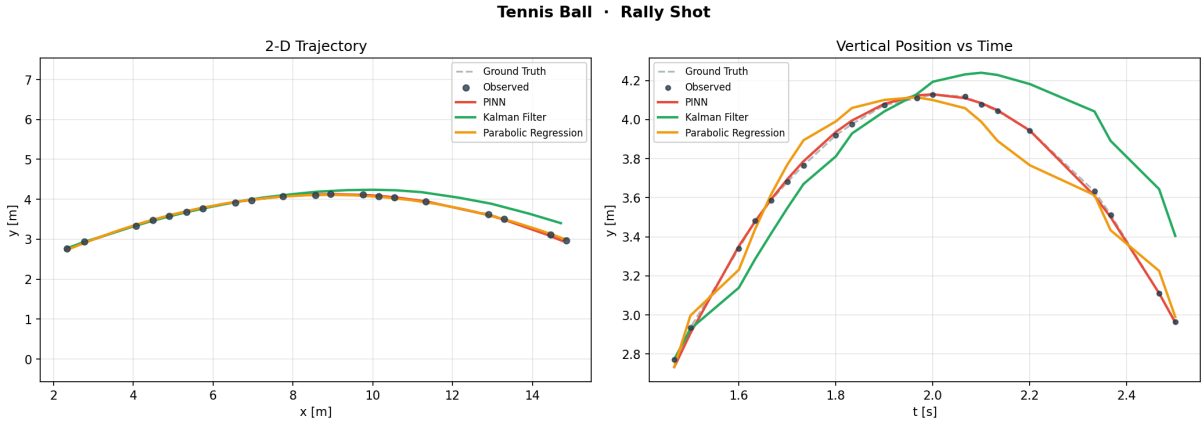


Figure 50: Trajectory comparison (PINN / Kalman / Parabolic) — Experiment 12

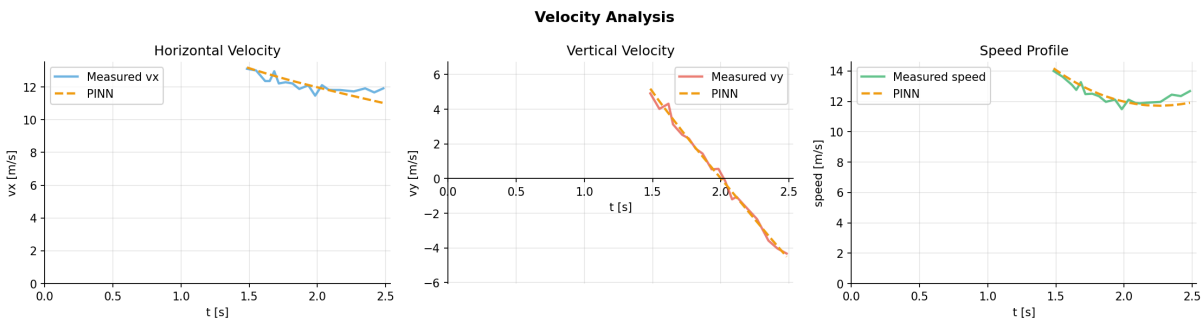


Figure 51: Velocity analysis — Experiment 12

Figure 52: Error metrics — Experiment 12

9 References

[1] Chiha, Z., Péteri, R., & Mascarilla, L. (2024). *Predicting 3D Projectile Motion in Table Tennis Using Computer Vision and Physics-Informed Neural Network*. CBMI 2024 — 21st International Conference on Content-Based Multimedia Indexing, Reykjavik, Iceland.

[2] Singh, P.R., Gottumukkala, R., & Maida, A. (n.d.). *An Analysis of Kalman Filter Based Object Tracking Methods for Fast-Moving Tiny Objects*. McNeese State University / University of Louisiana at Lafayette.

[3] Zhou, Y., et al. (2024). *Vision-Based 3D Reconstruction Methods Using Computer Vision* — a survey of current approaches to monocular and stereo-based scene reconstruction.

[4] Nielsen, N. *Monocular and Stereo Vision Pipelines for Object Detection and Visual Odometry* [Video tutorials]. YouTube. https://www.youtube.com/@NicolaiAI

[5] Kalman, R.E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1), 35–45. American Society of Mechanical Engineers (ASME).

[6] Fischler, M.A., & Bolles, R.C. (1981). Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Communications of the ACM*, 24(6), 381–395.

[7] Raissi, M., Perdikaris, P., & Karniadakis, G.E. (2019). Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations. *Journal of Computational Physics*, 378, 686–707.

[8] Welch, G., & Bishop, G. (2006). *An Introduction to the Kalman Filter*. Technical Report TR 95-041, University of North Carolina at Chapel Hill, Department of Computer Science.